



# Safeobuddy iOS SDK Documentation

## Table of Contents:-

1. [Implementation](#)
2. [Initialization](#)
3. [Authorization](#)
4. [Device List](#)
5. [Open Lock](#)
6. [Close Lock](#)
7. [Lock Records](#)
8. [Logout User](#)
9. [V7 Locks — Direct BLE Control](#)
10. [Response Codes](#)

# 1. Implementation

## Environment (minimum requirements)

- Xcode 14.0, iOS 13.0, Swift 5.0

**Device builds only.** SafeOBuddy.framework ships an arm64 device slice only. It does **not** contain a Simulator architecture, so the SDK can be built and tested on a physical iPhone only.

## Installation

- Add the following line to your app's Podfile:

```
pod 'SafeOBuddy', :git => 'https://github.com/DhanukaElectrotech/SafeOBuddy-iOS-SDK.git'
```

- Run `pod install`, then add the import to every file that uses the SDK:

```
import SafeOBuddy
```

## Permissions

- Ensure Bluetooth is enabled on the device.
- Add the following keys to your app's Info.plist:
  - Privacy — Bluetooth Peripheral Usage Description (NSBluetoothPeripheralUsageDescription)
  - Privacy — Bluetooth Always Usage Description (NSBluetoothAlwaysUsageDescription)

## Get Device List API

[https://vehicletrack.memnocool.com/ConService.aspx?method=vehdatawithlock&uid="USERNAME"&pwd="PASSWORD"&vno=](https://vehicletrack.memnocool.com/ConService.aspx?method=vehdatawithlock&uid=)

**Note on method names.** Two SDK symbols are spelled as shown below and must be typed exactly, or the code will not compile: `intializeSafeobuddy` (not *initialize*) and the closure argument label `complition` (not *completion*). Both are omitted when using Swift trailing-closure syntax, as in all examples in this document.

# 2. Initialization

- Initialize the SDK in your AppDelegate, inside `didFinishLaunchingWithOptions`:

```
Safeobuddy.intializeSafeobuddy { message in
    print("SDK initialized:", message)
}
```

# 3. Authorization

- Authorize the SDK after initialization:

```
Safeobuddy.authUser(email: "USERNAME", password: "PASSWORD") { message, statusCode in
    if statusCode == "600" {
        print("Authorization successful")
    } else {
        print("Authorization failed:", message)
    }
}
```

statusCode is a String in every callback, not an Int. Always compare against a quoted value — `statusCode == "600"`.

## 4. Device List

```
Safeobuddy.getDeviceList { response, message, statusCode in
    if statusCode == "600" {
        print("Device list:", response) // [[String: Any]]
    } else {
        print("Failed to fetch device list:", message)
    }
}
```

## 5. Open Lock

- Open a lock by device code:

```
Safeobuddy.openLock(deviceCode: "YOUR DEVICE CODE") { response, message, statusCode in
    if statusCode == "600" {
        print("Lock opened successfully:", message)
    } else {
        print("Failed to open lock:", message)
    }
}
```

- Open a lock when the lock data and MAC are already held by the app (skips the lookup call):

```
Safeobuddy.openLockWithData(deviceData: "LOCK DATA",
                            macId: "MAC ID",
                            deviceCode: "YOUR DEVICE CODE") { response, message, statusCode in
    print(statusCode, message)
}
```

## 6. Close Lock

```
Safeobuddy.closeLock(deviceCode: "YOUR DEVICE CODE") { response, message, statusCode in
    if statusCode == "600" {
        print("Lock closed successfully:", message)
    } else {
        print("Failed to close lock:", message)
    }
}
```

```
Safeobuddy.closeLockWithData(deviceData: "LOCK DATA",
                             macId: "MAC ID",
                             deviceCode: "YOUR DEVICE CODE") { response, message, statusCode in
    print(statusCode, message)
}
```

## 7. Lock Records

- Retrieve lock records (default: last 30 days):

```
Safeobuddy.getDeviceRecord(deviceCode: "YOUR DEVICE CODE") { response, message, statusCode in
    if statusCode == "600" {
        print("Lock records:", response)    // [[String: Any]]
    } else {
        print("Failed to fetch lock records:", message)
    }
}
```

- Retrieve lock records for a date range (MM/dd/yyyy):

```
Safeobuddy.getFilterDeviceRecord(deviceCode: "YOUR DEVICE CODE",
                                  fromDate: "MM/dd/yyyy",
                                  todayDate: "MM/dd/yyyy") { response, message, statusCode in
    if statusCode == "600" {
        print("Filtered lock records:", response)
    } else {
        print("Failed to fetch filtered lock records:", message)
    }
}
```

## 8. Logout User

```
Safeobuddy.logoutUser { message in
    print("User logged out:", message)
}
```

## 9. V7 Locks — Direct BLE Control

V7 locks are not driven through the lock-vendor cloud. The app talks to the lock directly over Bluetooth Low Energy: a 7-byte command packet is derived from the lock's MAC address and written to the lock's writable characteristic. This is the iOS counterpart of `manualLockAction(lockId, 71 / 70)` in the Android SDK.

### 9.1 Installation

The V7 path ships as a separate pod. Add it alongside the main pod:

```
pod 'SafeOBuddy', :git => 'https://github.com/DhanukaElectrotech/SafeOBuddy-iOS-SDK.git'
pod 'SafeOBuddyV7', :git => 'https://github.com/DhanukaElectrotech/SafeOBuddy-iOS-SDK.git'
```

```
import SafeOBuddyV7
```

Requires `NSBluetoothAlwaysUsageDescription` in `Info.plist`. No location permission is required on iOS.

### 9.2 Obtaining the Lock MAC

`macId` is the lock's Bluetooth MAC address. For V7 locks it is the **LockCode** field returned by the lock-MAC-details service:

<https://vehicletrack.memobocool.com/ConService.aspx?method=getlockmacdetails&lockid=<BT LOCK ID>&contactid=<USER ID>&token=<ACCESS TOKEN>>

The response carries `LockId`, `LockCode`, `MacId` and `Lockdata`. `<BT LOCK ID>` is the `btlockid` value from the device list (section 4).

**Use LockCode, not MacId.** For V7 locks the Bluetooth MAC is the `LockCode` field. The two fields are used the opposite way round for standard (non-V7) locks. Passing `MacId` to a V7 lock builds an incorrect command packet and the lock will not respond — and because the command is sent without acknowledgement, the call will still report success.

### 9.3 Open Lock

```
SafeobuddyV7.openLock(macId: "5C:53:10:4F:37:AE") { result in
    if result.isSuccess {                      // result.code == "106"
        print(result.message)                  // "Lock opened successfully."
    } else {
        print(result.code, result.message)
    }
}
```

### 9.4 Close Lock

```
SafeobuddyV7.closeLock(macId: "5C:53:10:4F:37:AE") { result in
    print(result.code, result.message, result.type)
}
```

To follow connection progress, pass an optional status handler:

```
SafeobuddyV7.openLock(macId: mac, onStatus: { status in
    print(status)    // "Connecting...", "Sending command...", ...
}) { result in
    print(result.message)
}
```

## 9.5 Command Packet

The packet is identical to the Android implementation. The MAC is byte-reversed, the first 6 bytes are taken, and the command byte is appended — 0x55 to open, 0x4C to lock:

|      |                      |
|------|----------------------|
| MAC  | 5C:53:10:4F:37:AE    |
| Open | AE 37 4F 10 53 5C 55 |
| Lock | AE 37 4F 10 53 5C 4C |

## 9.6 Locating the Lock

Android connects directly to a MAC address. iOS provides no equivalent API — CoreBluetooth never exposes a peripheral's MAC. The SDK therefore scans and matches the MAC against each advertisement (manufacturer data, service data and advertised name, in both byte orders). The first match is cached, so later opens connect immediately without scanning.

If the lock firmware does not publish its MAC in the advertisement, resolve the peripheral once through your own scan UI and register it:

```
SafeobuddyV7.manager.startScan { peripheral, advertisementData, rssi in
    // present the list and let the user select the lock
}

SafeobuddyV7.manager.register(macAddress: mac,
                              peripheralIdentifier: peripheral.identifier)
```

## 9.7 Using V7 Alongside the Standard Flow

The V7 module and the main SDK each drive an independent Bluetooth central manager. They do not share state, storage or delegates, but they do share the device radio. Nothing is started until a V7 method is called for the first time.

- Do not run a V7 action and a standard openLock / closeLock at the same time. Check SafeobuddyV7.isBusy first if both paths can be triggered from the same screen.
- Call SafeobuddyV7.shutdown() when leaving a V7 screen to release the radio back to the standard flow.

## 10. Response Codes

### A. Standard flow — `authUser`, `getDeviceList`, `getDeviceRecord`, `getFilterDeviceRecord`, `openLock`, `closeLock`

Every callback in the standard flow returns `statusCode` as a `String`.

| Status Code     | Description                                                                                                   |
|-----------------|---------------------------------------------------------------------------------------------------------------|
| "600"           | Success                                                                                                       |
| Any other value | Error. The accompanying <code>message</code> parameter carries the reason returned by the server or the lock. |

### B. V7 flow — `SafebuddyV7.openLock` / `closeLock`

The V7 result codes match the Android SDK's `onSafeLockAction` values exactly, so result handling can be shared between the two platforms.

| Status Code | Description                                                                                                                     |
|-------------|---------------------------------------------------------------------------------------------------------------------------------|
| 106         | Success Code: Success                                                                                                           |
| 100         | Error: Failed to lock the device for <code>type = "close lock"</code> , or invalid device information (missing / malformed MAC) |
| 102         | Error: Failed to open the lock for <code>type = "open lock"</code>                                                              |

The `type` field is `"open lock"` or `"close lock"`, matching Android.

### C. V7 result object

| Property               | Description                                                                 |
|------------------------|-----------------------------------------------------------------------------|
| <code>code</code>      | <code>String</code> — status code from the table above                      |
| <code>message</code>   | <code>String</code> — human-readable result message                         |
| <code>type</code>      | <code>String</code> — <code>"open lock"</code> or <code>"close lock"</code> |
| <code>isSuccess</code> | <code>Bool</code> — convenience for <code>code == "106"</code>              |